========== Shell scripts ==========

Q How to find a directory and then list the files inside it?

```
find . -type d -name "target_dir" -exec ls -l {} \;
```

Q How back up all .log files from a directory and move them to a backup folder with a timestamp?

```
#!/bin/bash
# Define source and backup directories
SRC_DIR="/home/user/logs"
BACKUP_DIR="/home/user/backup"
DATE=$(date +%Y%m%d)
# Create backup directory if it doesn't exist
mkdir -p "$BACKUP_DIR"
# Loop through each .log file in the source directory
for file in "$SRC_DIR"/*.log; do
    filename=$(basename "$file" .log)
    cp "$file" "$BACKUP_DIR/${filename}_$DATE.log"
    echo "Backed up $file to $BACKUP_DIR/${filename}_$DATE.log"
done
```

Q How to find all log files that have been modified within the last 5 hours?

```
find /path/to/search -name "*.log" -mmin -300
```

Q Write a shell script to Searches the pattern "error" from the log file created within 5hrs and generate a output filename:line_number:error_line_text ?

```
#!/bin/bash
SEARCH_DIR=${1:-.}                    # $1 is unset or empty, use . (current directory) as the default
TIME_RANGE_MINUTES=300
find "$SEARCH_DIR" -type f -name "*.log" -mmin -"$TIME_RANGE_MINUTES" | while read -r logfile; do
    # Search for 'error' in the file, case-insensitive, print filename, line number, and the line
    grep -i -n "error" "$logfile" | sed "s|^|$logfile:|"
done
```

Q Write a program to check the health of the server using python script?

```
import requests
servers = ['http://192.168.1.10', 'http://192.168.1.11']
for server in servers:
    try:
        r = requests.get(server, timeout=5)
        if r.status_code == 200:
            print(f"{server} is up!")
        else:
            print(f"{server} returned status: {r.status_code}")
    except requests.RequestException:
        print(f"{server} is down or unreachable.")
```

Q How to replace a partten permanently in a file?

```
sed –i ‘s/original_parten/replace_parten/‘ file_name
```

Q How to take the terraform.tfstate file backup using python?

```
import os
import shutil
from datetime import datetime

def backup_tfstate(tfstate_path='terraform.tfstate', backup_dir='tfstate_backups'):
    # Ensure the tfstate file exists
        if not os.path.isfile(tfstate_path):
                print(f"Error: '{tfstate_path}' not found.")
                return
    # Create backup directory if it doesn't exist
        os.makedirs(backup_dir, exist_ok=True)
    # Create a timestamped backup filename
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        backup_filename = f"terraform_{timestamp}.tfstate"
        backup_path = os.path.join(backup_dir, backup_filename)
    # Copy the tfstate file to the backup location
        shutil.copy2(tfstate_path, backup_path)
        print(f"Backup created: {backup_path}")
backup_tfstate()
```

========== git and jenkins ==========

Q Explain the Git Branching Strategy that you used in your company.

```
master
├── dev──────────feature
|       |             ├── feature1
|       ├──test       ├── feature2 ( merge f1 & f2 to dev branch ,do  testing , if is success create test branch )
|       ├── release  (create a tag/version, if stage env testing success go pre-prod deployment and testing )
|                      ( if pre-prod deployment and testing success merge to master )
├──────────────────deploy to production
|────hotfix ( if need to fix something in production)
```

Q Explain 3 challenges that you faced with Git during your work experience.

When multiple team members worked on the same feature or file, we frequently ran into merge conflicts.
To fix this we do frequent pulls and perform smaller commits

In feature branch --- git checkout feature

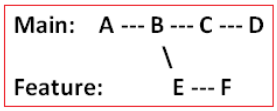
git add

git commit –m “ ---- ”

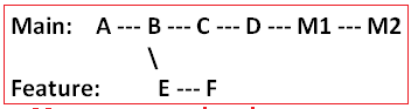
git pull origin master --rebase ---------- after every commit do this so your commit will be the latest commit and all master commit will be the previous

git push origin feature

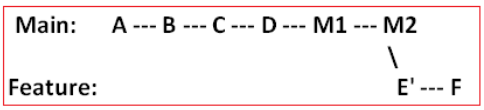
Encouraged the team to rebase often before merging to keep histories clean.



Initial state



Master moves ahead



on feature branch git pull origin master --rebase

Q Explain the recent challenge that you faced with Git and how did you address it? /

A teammate accidentally committed a Kubernetes Secret (base64 encoded) to Git. What you will do ?

ANS-> Alert the team that a secret has been exposed.

Find which commit introduced it ---> git log -p -S "<part-of-secret>"

Removes the file from all commits in Git history using git filter-repo

git filter-repo --path k8s/secret.yaml --invert-paths

then git push origin --force --all and git push origin --force --tags

next Everyone need to re-clone or hard reset their local repos.

Update the Kubernetes secret:

kubectl delete secret <secret-name>

kubectl create secret generic <secret-name> --from-literal=key=value

now Update the value wherever it's used (apps, CI/CD pipelines, etc.)
Replace credentials or regenerate API keys if the secret was cloud-based (e.g., AWS, GCP, Stripe).
Not to repeat the same error we install pre-commit package pip install pre-commit

Q Have you ever used Git tags ? If yes, why ?

Yes — I've "used" Git tags for Versioning , Deployment and rollback
git tag ---> to list all tags
git tag -a v1.0.0 -m "Release version 1.0.0 - stable production build" ----> create tag with message
git push origin v1.0.0 -----> push particular tag
git push origin --tags ------> push all tags

Pipeline code for below

1. **v1.2.0 is treated as a branch (not a tag).**
2. **It merges v1.2.0 → release.**
3. **Creates a tag (v1.2.0) on the merged release.**
4. **Builds & tests from the release branch.**
5. **If successful:**
   Publishes to JFrog Artifactory.
   Merges release → master → add a tag to it.

```
pipeline {
  agent any
  environment {
    GIT_CREDENTIALS_ID = 'git-credentials-id'       // Jenkins Git credentials ID
    JFROG_CREDENTIALS_ID = 'jfrog-credentials-id'   // Jenkins credentials ID for JFrog
    VERSION_BRANCH = 'v1.2.0'                       // Version branch
    RELEASE_BRANCH = 'release'                      // Release branch
    MASTER_BRANCH = 'master'                        // Master branch
    TAG_NAME = 'v1.2.0'                             // Tag to create after merge
    TAG_MESSAGE = 'Release 1.2.0 - stable production release'
    BUILD_TOOL = 'maven'                            // or 'gradle'
    ARTIFACT_PATH = 'target/*.jar'                  // Gradle: 'build/libs/*.jar'
    JFROG_URL = 'https://your.jfrog.io/artifactory/your-repo/' // Update with your Artifactory repo
  }
  stages {
```

```
stage('Checkout Version Branch') {
  steps {
    echo "Checking out branch ${VERSION_BRANCH}..."
    checkout([$class: 'GitSCM',
      branches: [[name: "*/${VERSION_BRANCH}"]],
      userRemoteConfigs: [[
        url: 'git@github.com:your-org/your-repo.git',
        credentialsId: "${GIT_CREDENTIALS_ID}"
      ]]
    ])
  }
}
stage('Merge Version to Release') {
  steps {
    echo "Merging ${VERSION_BRANCH} into ${RELEASE_BRANCH}..."
    sshagent (credentials: ["${GIT_CREDENTIALS_ID}"]) {
      sh """
        git fetch origin
        git checkout ${RELEASE_BRANCH}
        git pull origin ${RELEASE_BRANCH}
        git merge origin/${VERSION_BRANCH} --no-edit
        git tag -a ${TAG_NAME} -m "${TAG_MESSAGE}"
        git push origin ${RELEASE_BRANCH} --tags
      """
    }
  }
}
stage('Build') {
  steps {
    echo "Building project on ${RELEASE_BRANCH}..."
    script {
      if (BUILD_TOOL == 'maven') {
        sh 'mvn clean package -DskipTests=false'
      } else if (BUILD_TOOL == 'gradle') {
        sh './gradlew clean build'
      } else {
        error "Unsupported build tool: ${BUILD_TOOL}"
      }
    }
  }
}
stage('Test') {
  steps {
    echo "Running tests..."
    script {
      if (BUILD_TOOL == 'maven') {
        sh 'mvn test'
      } else if (BUILD_TOOL == 'gradle') {
        sh './gradlew test'
      }
    }
  }
}
stage('Publish to JFrog') {
  when {
    expression { currentBuild.currentResult == 'SUCCESS' }
  }
```

```
            steps {
              echo "Publishing artifact to JFrog Artifactory..."
              withCredentials([usernamePassword(credentialsId: "${JFROG_CREDENTIALS_ID}",
        usernameVariable: 'JFROG_USER', passwordVariable: 'JFROG_PASS')]) {
                sh """
                  echo "Uploading artifact to ${JFROG_URL}"
                  curl -u $JFROG_USER:$JFROG_PASS -T ${ARTIFACT_PATH} ${JFROG_URL}
                """
              }
            }
          }
          stage('Merge Release to Master') {
            when {
              expression { currentBuild.currentResult == 'SUCCESS' }
            }
            steps {
              echo "Merging ${RELEASE_BRANCH} into ${MASTER_BRANCH}..."
              sshagent (credentials: ["${GIT_CREDENTIALS_ID}"]) {
                sh """
                  git fetch origin
                  git checkout ${MASTER_BRANCH}
                  git pull origin ${MASTER_BRANCH}
                  git merge origin/${RELEASE_BRANCH} --no-edit
                  git tag -a ${TAG_NAME} -m "${TAG_MESSAGE}"
                  git push origin ${MASTER_BRANCH} --tags
                """
              }
            }
          }
        }
        post {
          success {
            echo " Build, test, publish, and merge pipeline completed successfully for ${TAG_NAME}."
          }
          failure {
            echo " Pipeline failed. Merge/publish steps skipped."
          }
        }
      }
```

Q How do you combine Multiple commits into a Single commit ?

git rebase -i HEAD~3 ---------- last 3 commit combine to single commite

Q How you detection conflict automatically, if there is no conflict merge the code?

Step 1 - Fetch latest branches ========> step 2 - Try merging dev into QA ========>

step 3 - Abort if conflict is detected

```
#!/bin/bash
set -e
# Setup
git checkout QA
git pull origin QA
git fetch origin dev
```

```
# Attempt to merge
    if git merge origin/dev --no-commit --no-ff; then
            echo "Merge successful, pushing to QA"
            git commit -m "Automated merge of dev into QA"
            git push origin QA
    else
            echo " Merge conflict detected between dev and QA!"
            git merge --abort
            # Optional: send a notification or create a GitHub issue
    fi
```

Q how to run two stages simultaneously in jenkins?

```
pipeline {
    agent any

    stages {
        stage('Parallel Stage') {
            parallel {
                stage('Stage A') {
                    steps {
                        echo 'Running Stage A'
                        // Your commands here
                    }
                }
                stage('Stage B') {
                    steps {
                        echo 'Running Stage B'
                        // Your commands here
                    }
                }
            }
        }
    }
}
```

Q I have 3 parallel jenkins stage if one will fail it should continue?

in the stage you mention the below

```
steps {
    catchError(buildResult: 'SUCCESS', stageResult: 'FAILURE') {
        echo 'Running Task A'
        sh 'exit 1' // simulate failure
    }
```

Q What are the stages you follow to create a build pipeline and deployment pipeline?

Build Pipeline

GitHub Source Code Checkout --------- Install Dependencies (mvn dependency:resolve) -------- Sonarqube check for coding standard (mvn sonar:sonar) --------- Compile/Build the code (mvn clean compile) --------- Run unit test and generate test report (junit 'test_ouput/focus_test_reports/*.xml') ---------- zip test report (zip -r test_output.zip test_output) -- Publish Test Report in HTML --------- Check Test Status percentage (> 95%) --------- Create Build Package (mvn package -DskipTests) -------- upload to jfrog articatory (archiveArtifacts artifacts: 'target/*.jar', fingerprint: true) --------- post stage (check test percentage and accordingly send the mail of success or failure)

========== Docker ==========

Q Port is Not Accessible on local host even after Port mapping in Docker

execute docker run -p 8080:80 myimage or docker-compose.yml

check errors from docker logs <container_id_or_name>

check docker ps

0.0.0.0:8080->80/tcp ----------> if output is not like this then your port mapping isn't in effect

Test Connectivity

curl http://localhost:8080 and from container curl http://localhost:80

Q What is the purpose of EXPOSE in Dockerfile ?

It tells users and other developers which ports the application expects to be accessed on.

EXPOSE 80 --------- in docker file

or docker run -d -p 8080:80 myimage manually in docker cli command

Q Docker Container Exits Immediately, how will you troubleshoot ?

docker ps –a ---------- get the container id

docker logs <container_id or name> ---> see the container output before exiting.

check the Command specified in CMD or ENTRYPOINT is valid or not

Q difference between CMD or ENTRYPOINT?

ENTRYPOINT – Defines the main executable that will always run.
CMD – Sets the default command to run when the container starts, but can be overridden at runtime.

```
FROM ubuntu
ENTRYPOINT ["echo"]
CMD ["Hello from CMD"]
```

docker run myimage ----------> Output: Hello from CMD

docker run myimage World ----------> # Output: World

EX 2

```
FROM python:3.11
ENTRYPOINT ["python"]
CMD ["app.py"]
```

----------> This will always run python app.py by default.

But if we do docker run myimage -m venv venv ----> it will execute python -m venv venv

Q You made change in your code, rebuilt the image, but the change isn't reflected?

Ensure your Dockerfile has copied your changed code

docker history your-image-name --------> You can inspect the built image to confirm layers

remove the image and recreate

Q Difference between copy and add in docker file

copy and add will copy the file from source to destination

but in add it also extract the file from tar file after copying

ex

Copy files into the container COPY app.py /app/app.py

Automatically extracts app.tar.gz into /app ADD app.tar.gz /app/

Q App Crashes with "Permission Denied" in Container but works fine on localhost?

check container log docker logs <container_id_or_name> or

docker exec -it <container_id_or_name> /bin/sh ---> cd /app/logs ---> tail -f app.log

Check File permissions Use chmod and chown if need to change the permissions

In dockerfile change the user to root (USER root) and try

Check the Volume mounts and check mapping permissions in docker-compose file

volumes:

- ./data:/app/data

Security configsConsider SELinux/AppArmor restrictions

-v ./data:/app/data:z

Q Docker host is running out of disk space. How do you clean up?

docker system df check the space used

docker system prune remove the Stopped containers , Unused networks , Build cache

docker system prune -a -f removes all unused images

docker volume ls and docker volume inspect <volume_name> Check volume usage

delete the contents of a Docker volume without deleting the volume itself, after archiving

```
docker run --rm \
        -v <volume_name>:/volume \
        -v $(pwd):/backup \
        alpine \
        tar  czf  /backup/<volume_name>.tar.gz   -C  /volume .

docker run --rm \
        -v <volume_name>:/volume \
        alpine \
        sh -c   "rm -rf /volume/*   /volume/.[!.]*   /volume/..?*  || true"
```

Q How to optimize a Java build image that's currently 1.5 GB, even though you're already using a lightweight base image? or what is Builder image?

1. Use a Multi-Stage Build (for Docker)

first dockerfile

```
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests
```

2nd dockerfile

```
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

COPY --from=builder  /app/target/your-app.jar .
```

2. exclude unnecessary dependencies, Logs or files from jar

Q How to filter stopped containers?

```
docker ps -a --filter "status=exited"
```

Q How will you Debug a Live Container ?

Check logs: `docker logs --tail <number_of_lines> -f <container_id_or_name>`

Inspect container metadata: `docker inspect <container_id_or_name>`

Q when will you forcefully remove a container and how ?

The container is stuck or unresponsive

```
kubectl get pods  # Look for CrashLoopBackOff, Error

kubectl describe pod <pod_name>  # See events and termination reason

kubectl logs <pod_name> --previous  # Get logs from the previous instance (crashed)
```

to clean up zombie containers

to remove container ---> docker rm -f <container_id_or_name>

========== Ansible ==========

Q How to create a hosts yaml file in Ansible?

The hosts.yaml file use to define the machines (hosts) that Ansible will manage and how to connect to them.

```
all:
  vars:
      ansible_user: ubuntu
      ansible_ssh_private_key_file: ~/.ssh/id_rsa
  children:
      webservers:
      hosts:
            web1.example.com:
            web2.example.com:
      dbservers:
      hosts:
            db1.example.com:
            ansible_host: 192.168.56.11
            ansible_port: 22
```

Give the folder structure for an ansible project?

```
My-deployment/          ----    The main project folder — everything related to automation lives here.
  ├── inventory/
  │   └── hosts.yml                 ----    Lists all your servers and groups (e.g., dev, test, prod).
  ├── group_vars/                   ----    Stores variables that apply to specific groups of servers.
  │   ├── all.yml                   ----    Common settings for all servers.
  │   ├── testservers.yml           ----    Settings only for test servers.
  │   └── devservers.yml            ----    Settings only for dev servers.
  ├── roles/                        ----    Holds reusable sets of tasks to modularize the automation
  │   └── pullmyparamfile/          ----    The folder tis specific to pulls parameter files
  │       ├── tasks/                ----    Actual code inside this folder
  │       │   ├── pullmyparamfile.yaml  ----  Code to pull parameter files
  │       │   └── main.yml          ----    entry point to run the task
  │       ├── vars/                 ----    variable only use in the role
  │       │     └── main.yml
  │       ├── templates/            ----    The folder contain the jinja file which has the variables
  │       │     └── mytempl.j2
  │       ├── files/                ----    this folder contain the static file or shell scripts
  ├── playbooks/                    ----    Defines what actions to run and where.
  │   ├── site.yml                  ----    the playbook which includes all other playbooks
  │   ├── deploy.yml                ----    Used to deploy applications.
  │   └── rollback.yml              ----    Used to undo a deployment.
  ├── files/                        ----    Stores raw files to copy directly to servers
  │   └── some_file.conf
  ├── templates/
  │   └── some_template.j2          ----    Stores template files (usually .j2 Jinja2 templates) that.
  ├── vars/                                       Ansible fills with variables before sending to servers
      └── global_vars.yml           ----    Holds global or project-wide variables.
```

Q How to run a play book in ansible?

ansible-playbook -i inventory/hosts.yml playbooks/deploy.yml -e "app_version=2.0.0 app_env=staging"

Q How to test the playbook is running as expected without changing anything?

ansible-playbook -i inventory/hosts.yml playbooks/deploy.yml --check

Q Ansible ensures repeated executions produce the same result by only applying changes when needed? /

Q How does Ansible handle idempotency?

Example --> If Nginx is already installed, the task is skipped. The Nginx will not reinstall.

```
- name: Ensure Nginx is installed
  apt:
     name: nginx
     state: present
```

Example --> If the user "sumanta" already exists, Ansible skips the task

```
- name: Ensure 'sumanta' user exists
  user:
       name: sumanta
       state: present
```

Example --> If the service is already running and enabled, Ansible skips the task.

```
- name: Ensure Nginx is running and enabled
  service:
      name: nginx
      state: started
      enabled: yes
```

Q What is the purpose of Ansible Handlers?

A handler task in Ansible will not execute on its own, it will only run if a previous task use notify.

Example --> If the config file changes, the copy task will notify the restart nginx handler. otherwise

the file didn't change, the handler is not run.

```
- name: Ensure Nginx config is present
        copy:
                src: nginx.conf
                dest: /etc/nginx/nginx.conf
        notify: restart nginx

handlers:
        - name: restart nginx
                service:
                        name: nginx
                        state: restarted
```

==========K8S ==========

Q Applicaton access in a networks range in k8s ?

Using ipBlock application access in a networks range

```
ingress:              ----  This ingress rule allows incoming traffic only from IP addresses inside the CIDR block
- from:
   - ipBlock:
       cidr: 192.168.0.0/16
```

Q how you can secure an application in k8s?

Use minimal base images

Scan images for vulnerabilities

Store sensitive data in Kubernetes Secrets.

```
kubectl create secret generic my-secret \
        --from-literal=username=admin \                  ------- create secrets from lietral
        --from-literal=password=secret123
                or

kubectl create secret generic my-secret \
--from-file=./mySecretFile.txt                           ------- create secrets from file
```

Set security contexts in k8 manifest file:

```
spec:
    securityContext:
        runAsNonRoot: true
    containers:
        - name: secure-container
          image: busybox
          command: [ "sh", "-c", "sleep 3600" ]
          securityContext:
                readOnlyRootFilesystem: true
                allowPrivilegeEscalation: false
                runAsNonRoot: true
```

Use NetworkPolicies to restrict communication between pods.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
        matchLabels:
              app: backend
  ingress:
        - from:
              - podSelector:
                    matchLabels:
                          app: frontend
  policyTypes:
          - Ingress
```

apply Ingress Security ---------- Advance topic

Use HTTPS for all traffic (TLS termination).

Enable Web Application Firewall (WAF) if supported (e.g., in cloud load balancers).

Set strict ingress rules to control which traffic reaches your services.

Add rate limiting, IP whitelisting, and header validation.

annotations:

nginx.ingress.kubernetes.io/ssl-redirect: "true" ---- Enforce HTTPS

nginx.ingress.kubernetes.io/limit-connections: "20" ---- Rate limiting to protect abusing the system (ex Only allow 20 requests per minute from each user.

nginx.ingress.kubernetes.io/whitelist-source-range: "203.0.113.0/24" ---- Only devices with those IP addresses/ in group can access it

Q If the pod not getting configured in node what will be the cause?

The Node doesn't have enough CPU or memory for the Pod's requests.

kubectl describe pod <pod-name> ------ It may show something like 0/3 nodes are available: 3 Insufficient cpu.

The Pod's spec uses nodeSelector, nodeAffinity, or taints/tolerations that don't match any Node

kubectl describe pod <pod-name> ------- It may show something like 0/3 nodes are available: 3 node(s) didn't match node selector.

So Verify your Pod spec:

```
spec:
    nodeSelector:
        disktype: ssd
```

Make sure the Node has the correct label

kubectl get nodes --show-labels

The Node is tainted, and the Pod doesn't have a toleration.

kubectl describe node <node-name> ----- It may show something like
Taints: node role.kubernetes.io/master:NoSchedule

For toleration we can Add a toleration to your Pod spec

```
tolerations:
  - key: "node-role.kubernetes.io/master"
    effect: "NoSchedule"
```

The Pod is created but can't pull the container image or connect to the registry.

kubectl get pods

kubectl describe pod <pod-name> ----- It may show something like Failed to pull image "nginx:latest": image pull back-off

To fix this we need to Verify the image name and registry credentials. and Ensure Node has network access to registry.

The Node itself is not in a Ready state.

kubectl get nodes

To fix this we need to Verify network connectivity and kubelet service.

Check the Security or policy tools (like OPA Gatekeeper, Kyverno, or PodSecurityPolicy) are rejecting the Pod.

kubectl describe pod ---- Error from server (Forbidden): admission webhook "validation.gatekeeper.sh" denied the request

To fix Adjust Pod spec or Security policy rules.

Final code for pod spec will be as below

```
apiVersion: v1
kind: Pod
metadata:
  name: example-debug-pod
  labels:
    app: debug-app
spec:
  # Ensure resources fit on available nodes
  containers:
  - name: debug-container
    image: nginx:1.25-alpine   # reliable image to avoid image pull errors
    resources:
      requests:
        cpu: "100m"            # low CPU request to avoid "Insufficient CPU"
        memory: "128Mi"        # low memory request
      limits:
        cpu: "200m"
        memory: "256Mi"
    securityContext:
      runAsNonRoot: true
      allowPrivilegeEscalation: false
      capabilities:
        drop: ["ALL"]
      readOnlyRootFilesystem: true
      seccompProfile:
        type: RuntimeDefault
  # Node selection - only schedule on nodes with disktype=ssd
  nodeSelector:
    disktype: ssd
  # Toleration - allow scheduling on master/control-plane nodes if needed
  tolerations:
    - key: "node-role.kubernetes.io/master"
      operator: "Exists"
      effect: "NoSchedule"
  # Avoid OPA/Gatekeeper rejects - no privileged fields, uses PSS restricted
  terminationGracePeriodSeconds: 10
  # Optional: Avoid imagePullBackOff if using private registries
  imagePullPolicy: IfNotPresent
```

Q how to run a pod in a particular node?

nodeAffinity and preferredDuringSchedulingIgnoredDuringExecution and requiredDuringSchedulingIgnoredDuringExecution

Q what is readenessprob and livenessProbe?

Liveness Probe checks if the container is still running. If it fails, the container will be restarted.
Readiness Probe checks if the container is ready to handle traffic.
If it fails, K8s will stop routing traffic to that pod. so the pod is removed from the service endpoints (url).

```
apiVersion: v1                  # API version for core Kubernetes objects like Pods
kind: Pod                       # Specifies this object is a Pod
metadata:
  name: affinity-probes-demo    # Name of the Pod for identification
  labels:
    app: demo                   # Label used for selectors, grouping, or filtering

spec:
  containers:                   # List of containers that will run inside this Pod
    - name: web                 # Name of the container
      image: nginx:1.25-alpine  # Container image; Alpine version is lightweight and stable
      ports:
        - containerPort: 80     # Declares the port the container listens on (useful for probes/services)

      # -------------------------
      # LIVENESS PROBE
      # -------------------------
      livenessProbe:            # Checks if the container is healthy; restarts it if failing
        httpGet:                # Uses HTTP GET request for health check
          path: /               # Path to probe inside the container
          port: 80              # Port where the probe will check
        initialDelaySeconds: 10 # Wait 10 seconds before running first liveness probe
        periodSeconds: 10       # Run liveness probe every 10 seconds
        timeoutSeconds: 2       # Probe must respond within 2 seconds or it's considered failed
        failureThreshold: 3     # After 3 failed checks, Kubernetes restarts the container
      # -------------------------
      # READINESS PROBE
      # -------------------------
      readinessProbe:           # Checks if the container is ready to receive traffic
        httpGet:                # Uses HTTP GET request
          path: /               # Path to check readiness
          port: 80              # Same port as the container
        initialDelaySeconds: 5  # Start readiness checking 5 seconds after container starts
        periodSeconds: 5        # Run readiness probe every 5 seconds
        timeoutSeconds: 2       # Must respond within 2 seconds
        failureThreshold: 3     # After 3 failures, container is marked NotReady (not restarted)

  # -------------------------
  # NODE AFFINITY SETTINGS
  # -------------------------
  affinity:
    nodeAffinity:               # Controls which nodes Pod should run on based on node labels

      # REQUIRED node affinity (hard rule ---- is a condition that must be satisfied for the Pod to be scheduled )
      requiredDuringSchedulingIgnoredDuringExecution:
        # If the requirement is not met, Pod will stay Pending
        nodeSelectorTerms:      # A list of possible node selector conditions
          - matchExpressions:   # Defines expressions to match node labels
              - key: disktype   # Node must have this label key
                operator: In    # Operator 'In' means value must match one in this list
                values:
                  - ssd         # Only schedule on nodes where disktype=ssd
      # PREFERRED node affinity (soft rule ---- if no such node exists, the Pod will be scheduled somewhere else )
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 50            # Priority weight; higher = more preferred
          preference:
            matchExpressions:
              - key: region     # Node label key to check
                operator: In    # Value must match one in the list
                values:
                  - us-east     # Prefer nodes in region=us-east

        - weight: 30            # Second preference with lower priority
          preference:
            matchExpressions:
              - key: environment # Another node label key
                operator: In    # Again expecting a matching value
                values:
                  - production   # Prefer nodes labeled environment=production
```

Q what is the issue if a Kubernetes Service port is enabled but you can't access the Pod?

If a Kubernetes Service port is enabled but you can't access the Pod, it usually means there's a networking, configuration, or selector mismatch issue between the Service, Pod, and Network setup.

To debug the issue we need to check the selector , pod label , service target port , network type and service type.

1-> check if the service selector is matching to pod label or not

kubectl get svc <service-name> -o yaml
kubectl get pods -l <selector-labels>
kubectl get endpoints <service-name> ---- If ENDPOINTS is empty, Service isn't connected to any Pods.

selector: ----- in service yaml file
app: my-app
----
labels: --------- in Pod/Deployment yaml file
app: my-app

2-> Check if the Pod is listening on the Service's targetPort or not

kubectl describe pod <pod-name> | grep -A2 "Ports"
kubectl describe svc <service-name>

Ensure the targetPort in Service matches the container port:

```
ports:
  - port: 80
    targetPort: 8080     # must match containerPort
```

3-> Check if the used network policy is blocking the traffic from the service or network

kubectl get networkpolicy -A
kubectl describe networkpolicy <policy-name> -n <namespace> ----- inspect each networkpolicy

To fix the networkpolicy you can allow certain traffic as per our requirement.

If we want to allow traffic only from pods labeled role=frontend to pods labeled role=backend

```
apiVersion: networking.k8s.io/v1      # API group for NetworkPolicy resources
kind: NetworkPolicy                   # This object defines network traffic rules
metadata:
  name: allow-frontend-to-backend     # Name of the NetworkPolicy
  namespace: my-app                   # Namespace where this policy applies

spec:
  podSelector:
    matchLabels:
      role: backend                   # This policy applies to Pods labeled role=backend

  policyTypes:
    - Ingress                         # This policy defines inbound traffic rules

  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend          # Only Pods with role=frontend can reach backend
      ports:
        - protocol: TCP               # Allow TCP traffic only
          port: 8080                  # Allow only TCP port 8080 into backend Pods
```

When we want to allow backend pods / a pod to call an external database or API by IP or CIDR

```
apiVersion: networking.k8s.io/v1        # API group for network policies
kind: NetworkPolicy                     # Declares this resource is a NetworkPolicy
metadata:
  name: allow-dns                       # Policy name
  namespace: my-app                     # Policy applies only in the my-app namespace

spec:
  podSelector: {}                       # Selects ALL Pods in this namespace (my-app)

  policyTypes:
    - Egress                            # This policy applies to outgoing traffic

  egress:
    - to:
        - namespaceSelector:            # Select the kube-system namespace
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:                  # AND select Pods inside kube-system namespace
            matchLabels:
              k8s-app: kube-dns         # Specifically kube-dns CoreDNS Pods

      ports:
        - protocol: UDP                 # Allow DNS UDP queries
          port: 53
        - protocol: TCP                 # Allow DNS over TCP (fallback)
          port: 53
```

4-> If you are trying to accesss the service from outside the cluster then check the service type.

because the ClusterIP → only accessible inside the cluster.

NodePort or LoadBalancer → accessible from outside

kubectl get svc <service-name> ----> check the service type

To fix this change the service type if needed

kubectl patch svc <service-name> -p '{"spec": {"type": "NodePort"}}'

5-> Check the Pod is in Ready state to receive traffic.

kubectl get pods -o wide

kubectl describe pod <pod-name>

kubectl logs <pod-name>

Check your Deployment or Pod YAML for a readinessProbe. If the readiness probe fails, Kubernetes removes the pod from the Service load balancer, meaning it won't receive traffic until it passes again.

To fix this you can Increase initialDelaySeconds, timeoutSeconds, or failureThreshold.

```
readinessProbe:
    httpGet:
        path: /healthz
        port: 8080
    initialDelaySeconds: 10
    periodSeconds: 5
```

6-> Check if there is problems with CNI (Calico, Flannel, etc.) can block traffic.

kubectl get pods -n kube-system

kubectl logs -n kube-system <cni-pod>

To fix this Restart the CNI plugin or verify network configuration.

kubectl rollout restart daemonset <cni-daemonset-name> -n kube-system
(<cni-daemonset-name> --> calico-node / kube-flannel-ds / cilium / weave-net)
kubectl rollout status daemonset <cni-daemonset-name> -n kube-system
kubectl get pods -n kube-system -l k8s-app=<cni-name> ----- to confirm all CNI pods are healthy

Q how to rollback k8 deployment from jenkins?

sh ''' kubectl rollout undo deployment/your-deployment-name --to-revision=2 -n your-namespace '''

Q what is the root cause if pod is not in ready state and how you will fix it?

or

Q What is the root cause for Kubernetes pod is in a CrashLoopBackOff?

If the pod is not in ready state then the Root Causes may be

Ensure the app exposes the correct port or path

Ensure the pod should have readinessProbe , if ruined increase initialDelaySeconds, timeoutSeconds, or failureThreshold.

Verify the image name and tag is correct

verify the pod should not face CrashLoopBackOff ------ means The container starts but immediately exits.

Check environment variables or ConfigMaps/Secrets is correct

Check the node resources (cpu and memory) is available to run the pod

Ensure volume mount paths and names match.

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: myapp-deployment
labels:
        app: myapp
spec:
replicas: 2
selector:
        matchLabels:
        app: myapp
template:
        metadata:
        labels:
                app: myapp
        spec:
        containers:
                - name: myapp-container
                image: myregistry/myapp:1.0.0   # Verify this image name and tag exist
```

```
        imagePullPolicy: IfNotPresent
        ports:
            - containerPort: 8080       # Ensure app actually listens on this port
        env:
            - name: APP_ENV
            value: "production"
            - name: CONFIG_PATH
            value: "/app/config/config.yaml"
            # If sensitive data, use Secrets/ConfigMaps
            # - name: DB_PASSWORD
            #   valueFrom:
            #     secretKeyRef:
            #       name: myapp-secret
            #       key: db_password

        # Readiness Probe: ensures traffic is only sent when the app is ready
        readinessProbe:
            httpGet:
            path: /healthz/ready
            port: 8080
            initialDelaySeconds: 10
            periodSeconds: 5
            timeoutSeconds: 3
            failureThreshold: 5

        # Liveness Probe: restarts pod if it becomes unhealthy
        livenessProbe:
            httpGet:
            path: /healthz/live
            port: 8080
            initialDelaySeconds: 20
            periodSeconds: 10
            timeoutSeconds: 3
            failureThreshold: 3

        # Resources: ensure sufficient node capacity and prevent overcommit
        resources:
            requests:
            cpu: "250m"
            memory: "256Mi"
            limits:
            cpu: "500m"
            memory: "512Mi"

        # Volumes: verify mount path and volume name match below
        volumeMounts:
            - name: config-volume
              mountPath: /app/config

# Volume definition
volumes:
        - name: config-volume
        configMap:
            name: myapp-config   # Must match an existing ConfigMap name
```

```
# Service to expose the Deployment
apiVersion: v1
kind: Service
metadata:
name: myapp-service
spec:
selector:
        app: myapp
ports:
        - protocol: TCP
        port: 80
        targetPort: 8080
type: ClusterIP
```

Q What could cause brief application downtime during a rolling update in Kubernetes, and how can prevented it in CI/CD pipeline?

or

Q What is the root casue for Users experience intermittent failures when new pods were being spin up and old ones shut down.

The root cause can any of the below

Readiness Probes were misconfigured or missing:

New pods were marked as "ready" before the app was fully initialized. As a result, traffic was routed too early, causing failures.

Old pods were terminated too quickly:

Kubernetes killed old pods before new ones were ready to serve traffic. Caused capacity dips during the rollout.

Validation / check the log

Monitoring showed 5xx errors during deployment windows.

```
kubectl  describe  deployment  <deployment-name>
kubectl  get  events  --sort-by=.metadata.creationTimestamp
```

see if there is any pods failing readiness probes, crash loops, or failed scheduling error

```
kubectl rollout status deployment <deployment-name> --watch
kubectl  get pods  -l  app=<app-label>  -w
```

See if pods go through Running → Terminating → Running too quickly.

```
kubectl  logs  <pod-name>  --previous
```

Look for stack traces, connection failures, or init errors.

Load balancers temporarily routed traffic to unhealthy pods.

```
kubectl  get  pods  -l  app=<your-app-label>  -w  ----> Pods go from Running but not Ready (0/1 Ready).
kubectl  describe  pod  <pod-name>
```

Look at: Readiness probe events , Start and readiness timestamps ,Failures in readiness checks

== Ingress/controller logs (NGINX, ALB, etc.): ==

For NGINX -----> kubectl logs -n ingress-nginx <nginx-ingress-pod> | grep upstream

Look for: "upstream connect error" ,"no live upstreams" ,"upstream timed out"

See if upstream targets became unhealthy during rollout.

For ALB -------> kubectl get ingress -A and kubectl get svc -A

each Ingress or Service of type LoadBalancer creates a corresponding ALB and Target Group.

aws elbv2 describe-load-balancers | grep <alb-name> # Find ALB name

aws elbv2 describe-target-groups --load-balancer-arn <alb-arn>

aws elbv2 describe-target-health --target-group-arn <target-group-arn>

kubectl get pod -o wide

---------- Solution ----------

Configured Proper Readiness Probes:

Used maxUnavailable and maxSurge in the rolling update strategy to control how many pods can be down or started during an update:

Make sure Kubernetes doesn't stop old pods until new ones are running and ready.

Add graceful shutdown logic in the application code.

The app handles the SIGTERM signal when Kubernetes stops a pod.

Finished (in-flight requests)current requests before shutting down

Add a pre-Stop hook in the pod spec to delay termination.

Give a buffer time to application to shut down cleanly before the pod is removed

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: my-app
labels:
        app: my-app
spec:
replicas: 3
strategy:
        type: RollingUpdate      # RollingUpdate for a pod
        rollingUpdate:
        maxUnavailable: 0        # Ensures no pod is down during updates
        maxSurge: 1              # Only one extra pod above desired count during update
selector:
        matchLabels:
        app: my-app
template:
        metadata:
        labels:
                app: my-app
```

```
spec:
terminationGracePeriodSeconds: 30   # Allow app time to shut down gracefully
containers:
- name: my-app
        image: myregistry/my-app:latest
        ports:
        - containerPort: 8080
        readinessProbe:                                          # chek the pod is ready
        httpGet:
                path: /healthz
                port: 8080
        initialDelaySeconds: 5                  # check the heart beet of the pod periodically
        periodSeconds: 10
        timeoutSeconds: 3
        successThreshold: 1
        failureThreshold: 3
        lifecycle:
        preStop:
                exec:
                command: ["/bin/sh", "-c", "echo 'Received SIGTERM, waiting for connections
to close...'; sleep 10"]
                # sleep 10 gives time to finish in-flight requests before termination
        env:
        - name: APP_ENV
        value: "production"
        - name: GRACEFUL_SHUTDOWN
        value: "true"
```

Q What are the storage array you worked on?

We work on AWS EBS (elastic block storage), S3
io2 type EBS --> Critical business applications for databases like Oracle, SQL Server
gp3 type EBS --> For development/test environments / Web servers / App servers

Q how you create volume where the pod will run in same AZ (availability zone)?

Use volumeBindingMode: WaitForFirstConsumer ----> it will create volume where the pod will be scheduled.
if you use volumeBindingMode: Immediate ----> it create volume without knowing where pod will be scheduled.

Q What is reclaimPolicy and how it is used in k8s? / How to manage lifecycle storage in cloud?

reclaimPolicy defines what happens to the PV when the PVC is deleted. So reclaimPolicy manage lifecycle for storage.

reclaimPolicy: Retain or reclaimPolicy: Delete
reclaimPolicy: Delete --> when you delete the PersistentVolume (PV)the data can not be retain
reclaimPolicy: Retain --> it keeps the PersistentVolume (PV) and delete the PVC so the data can be retain

Q What is storageClass in k8?

StorageClass defines the type of storage that can be dynamic provisioning of storage.
StorageClass defines the type of storage dynamically using
reclaimPolicy and volumeBindingMode
There is different storageClassName (fast, slow, standard) provided by the cloud provider.
storageClassName: fast --> usually refers to the storage with better performance , like SSD-backed volumes
storageClassName: slow --> this refers Cheaper, lower performance disks
storageClassName: standard --> general-purpose storage (e.g., magnetic or standard HDD)

Q How you can provision the storage dynamicaly in Kubernetes?

Define a StorageClass with a specific provisioner (e.g., AWS EBS)
Create PVC using that StorageClass.
Kubernetes automatically provisions a PV that matches the PVC using the StorageClass's configuration.
The PVC is bound to the newly created PV
A pod mounts the volume using the PVC.

Define storageClass

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
    name: fast
provisioner: kubernetes.io/aws-ebs   # Or use CSI like ebs.csi.aws.com for containerized workloads
parameters:
    type: gp3
    fsType: ext4
    iops: "6000"            # Custom IOPS (minimum: 3000, maximum: 16000)
    throughput: "250"       # MB/s (max: 1000)
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true  # Enables `kubectl edit pvc` to resize
```

create PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
    name: mypvc
spec:
    accessModes:
        - ReadWriteOnce
resources:
    requests:
        storage: 10Gi
storageClassName: fast
```

Use the PVC in a Pod

```
apiVersion: v1
kind: Pod
metadata:
    name: mypod
spec:
    containers:
        - name: app
          image: nginx
          volumeMounts:
              - mountPath: "/usr/share/nginx/html"
                name: myvolume
    volumes:
        - name: myvolume
          persistentVolumeClaim:
          claimName: mypvc
```

Q Your team deployed a PersistentVolumeClaim (PVC) on EKS, but it remains in Pending state. The storageClassName is set to gp2. What could be the issue?

check available storage classes --- kubectl get storageclass

AWS deprecated gp2. So we need to use gp3 storageClassName or create a gp2 StorageClass manually

Q Your pod fails to start due to an EBS volume mounting error. Logs show AZ mismatch. What's wrong?

EBS volume created in one AZ cannot be attached to a node in another AZ.

So ensure your cluster uses WaitForFirstConsumer in the StorageClass.

volumeBindingMode: WaitForFirstConsumer

Q You need to retain an EBS volume for manual backup even if a PVC is deleted. How can this be achieved via StorageClass?

Use reclaimPolicy: Retain in your StorageClass

Q You're upgrading your cluster to use the AWS EBS CSI driver instead of the in-tree provisioner. What changes must be made to StorageClasses?

Update your StorageClasses to use the new CSI provisioner

provisioner: ebs.csi.aws.com

Q On EKS you created a new StorageClass gp3-fast but PVCs still use gp3. How can you change the default StorageClass?

Patch the new class to be default

kubectl patch storageclass gp3-fast -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

========== AWS & Terraform ==========

Q What are Policies?

what actions are allowed or denied on which AWS resources. Policy use to grant user access to resources/services

Q What are Roles?

A role uses policies to define what it can do. Role use to grant access between two services/resources

Q how you can insert ec2 from one region to another region in aws?

In the source region where the EC2 instance already exist:

Go to the EC2 Dashboard. --> Select the instance you want to move. --> Click Actions

--> Image and templates --> Create image. --> Fill in the image name and click Create image.

The image (AMI) which you have created copy the AMI to the target region

Go to EC2 → AMIs in the source region --> Select your AMI → Actions → click Copy AMI

--> Choose the destination region --> Rename it --> Click Copy AMI

Launch EC2 in the new region using the coped AMI

If you need certain volumes (EBS) then follow below as well,

you can Create a snapshot of the Source volume --> Copy the snapshot to another region --> Create a new volume from that snapshot. ---> Attach it to a new instance.

Go to the AWS EC2 Console ---> In the left-hand menu, click Volumes under "Elastic Block Store" --->

Select the volume attached to the source EC2 instance --> Click Actions → Create snapshot --> Fill in a name --> create

To Copy the Snapshot to Another Region ---> Go to Snapshots in the EC2 menu (under "Elastic Block Store") --> Select the snapshot you just created --> Click Actions → Copy snapshot --> Choose the destination region --> Click Copy snapshotCreate a New Volume from the Snapshot

Create a New Volume from the Snapshot --> go to EC2 → Snapshots --> Find the copied snapshot -->

--> click Actions → Create volume ---> Choose the Availability Zone where your new EC2 instance is

---> Select the volume type and size if needed → Create volume

Attach the New Volume to a New (or Existing) EC2 Instance ---> Go to EC2 → Volumes --> Select the new volume --> Click Actions → Attach volume --> Choose the EC2 instance to attach it to --> Choose a device name (/dev/sdf)--> Click Attach volume.

Q where terrafrom is installed in your organization? and how you are accessing it?

Terraform is installed on a remote internal server.

To access it login to jump server ---> login remote machine ---> access terrafrom (using terrafrom init / plan /apply)

Q What is ploicies in AWS?

Policies in AWS define the permissions what actions are allowed or denied on which resources (Ec2 , S3).

Q How to create policy in terraform?

Step 1 --> Create a datasource which use to retrive data from AWS

```
data "aws_iam_policy_document" "s3_read_only" {
    statement {
        effect = "Allow"

        actions = [
            "s3:GetObject",
            "s3:ListBucket"
        ]

        resources = [
            "arn:aws:s3:::my-example-bucket",
            "arn:aws:s3:::my-example-bucket/*"
        ]
    }
}
```

Step 2 --> create a policy which can be used for a role or user

```
resource "aws_iam_policy" "s3_read_only" {
    name   = "S3ReadOnlyPolicy"
    policy = data.aws_iam_policy_document.s3_read_only.json
}
```

Q what is IAM role?

An IAM role is a set of permissions in AWS that defines what actions are allowed or denied for a trusted entity such as a user, service(ec2), or application to perform specific actions on AWS resources

Q How to create a Role in IAM for Account A in AWS?

AWS Console → IAM → Roles → Create role → Choose trusted entity (ec2 , S3) → Attach permissions policies (S3ReadOnly , get) → Add tags (Key = Environment, Value = Production) → Name the role (ex-: EC2S3AccessRole) → Create role

Q How to create Role using terraform? / How to attach this policy to a role or user?

```
resource "aws_iam_role" "ci_cd_role" {
    name = "ci-cd-deploy-role"
    assume_role_policy = data.aws_iam_policy_document.assume_role_policy.json
}
```

Q What is IAM USER?

IAM users are individual identities in AWS assigned with credentials and permissions to access AWS services and resources securely

Q How to create IAM user in AWS? How to create user access key and secret key in AWS?

AWS console ---- IAM ---- add user ---- provide user name , aws access type --- attach policy ---- provide tag ---- click on create user

Now you will get the user name, access key, secret access key for that user and you can download the csv fie and save it.

Q How to enforce least privilege and secure access without manual intervention for user?

step 1--> create user

```
resource "aws_iam_user" "user" {
        name = "least-privilege-user"
        force_destroy = true  # Allows destroying user with keys attached
}
```

step 2 --> create policies and impose policy to the user

```
resource "aws_iam_user_policy" "readonly_s3" {
        name = "readonly-s3-policy"
        user = aws_iam_user.user.name
        policy = data.aws_iam_policy_document.s3_read_only.json
}
```

step 3 --> generating access keys via Terraform

```
resource "aws_iam_access_key" "user_key" {
        user = aws_iam_user.user.name
}
```

step 4 --> Enforce password reset at next login if you want console access

```
resource "aws_iam_user_login_profile" "login" {
        user   = aws_iam_user.user.name
        password_reset_required = true
        pgp_key  = "keybase:username"  # encrypt password automatically
}
```

Q How do you enforce least privilege (Minimal Permissions)access control in IAM AWS.

1 Apply permissions (s3:GetObject, ec2:StartInstances)only to specific resources (e.g., one S3 bucket, not all buckets)

2 Assign role to the user

3 Different roles for differnt activity (one role manages infrastructure, another deploys applications)

Q How can IAM policy deployment be terraform?

create IAM policy in Terraform using aws_iam_policy.

```
resource "aws_iam_policy" "example_policy" {
        name       = "MyExamplePolicy"
        description = "A test policy created by Terraform"
        policy     = data.aws_iam_policy_document.s3_read_only.json
}
```

Q How can IAM policy deploy as part of infrastructure code?

In terraform we can use " aws_iam_policy " for IAM policy.

```
resource "aws_iam_policy" "example_policy" {
     name       = "DescribeEC2Policy"
     description = "Allows describing EC2 instances"
     policy     = file("${path.module}/policy.json")
}
```

uses tags (attributes) to control access to resources

Q What is IAM Identity Center?

IAM Identity Center formerly known as AWS Single Sign-On to AWS accounts.

Q How to enable centralized user provisioning automatically?

Automated access can be managed using:

SCIM (System for Cross-domain Identity Management)

Attribute-based access control (ABAC) ----> uses tags (attributes --> Department = "Engineering") to control access to resources (ec2, S3)

Pre-configured permission sets mapped to roles across accounts ----> The company has many departments /accounts like Finance ,IT, HR and some departments (like IT) have sub-teams, like: IT Support , IT Development

Q What is (SCIM) System for Cross-domain Identity Management and how you could do it in terraform?

For an example your HR system (IT team) already keeps a list of who works at the company.

But you want all those people get access to AWS services automatically also update the access when someone is hired, leaves, or changes roles.

```
aws-iam-hr-sync/
        |
        ├── scripts/
        |     └── hr_sync.py          # Script to pull user data from HR system
        |
        ├── data/
        |     └── example_hr_data.json   # Sample HR data (for dev/testing , team add data to property file)
        |
        ├── main.tf               # Main Terraform config
        ├── variables.tf          # Variable definitions
        ├── outputs.tf            # Output definitions
        ├── provider.tf           # AWS provider setup
        |
        ├── iam/
        |     ├── groups.tf          # IAM groups and policies per role
        |     ├── users.tf           # IAM user creation logic
        |     └── memberships.tf       # IAM user-to-group mapping
        |
        ├── terraform.tfvars        # Values for variables (e.g., region)
        ├── .gitignore
        └── ci/cd
              └── Jenkinsfile     # (Or Jenkins, etc.) CI/CD automation
```

===== hr_sync.py ======

```
import json

# Simulate HR system API
employees = [
        {"username": "john.doe", "role": "developer"},
        {"username": "jane.smith", "role": "hr"},
        {"username": "jane.smith", "role": "IT"}
]
```

```
# Transform to desired format / as a dictionary
output = {}
for idx, emp in enumerate(employees, start=1):
    key = f"user{idx}"
    output[key] = {
        "username": emp["username"],
        "role": emp["role"]
    }

# Write JSON to a file
with open("result.json", "w") as f:
    json.dump(output, f, indent=4)
```

==== example_hr_data.json ====

```
{
  "users": {
    "john.doe": {
      "role": "developer"
    },
    "jane.smith": {
      "role": "hr"
    }
  }
}
```

====main.tf==== external data source from scripts/hr_sync.py

```
data "external" "hr_users" {
  program = ["python3", "${path.module}/scripts/hr_sync.py"]
}
```

====variables.tf==== variables like region, default policy ARNs

```
variable "default_policy_arn" {
  type    = string
  default = "arn:aws:iam::aws:policy/ReadOnlyAccess"
}
```

==== groups.tf==== Defines IAM groups for roles like developer, hr, admin

```
resource "aws_iam_group" "developer" {
  name = "DeveloperGroup"
}
```

==== users.tf ==== Creates IAM users dynamically based on HR data

```
resource "aws_iam_user" "users" {
  for_each = data.external.hr_users.result["users"]
  name = each.key
}
```

==== memberships.tf ====== Maps IAM users to the appropriate IAM group

```
resource "aws_iam_user_group_membership" "group_membership" {
  for_each = data.external.hr_users.result["users"]
  user   = aws_iam_user.users[each.key].name
  groups = [aws_iam_group.${each.value["role"]}.name]
}
```

Q How to provide access to the resources to the users in IAM?

we can provide access to the resource using Attribute-Based Access Control (ABAC).

Step 1 --> Tag IAM Users with Department Attribute

```
resource "aws_iam_user" "developer" {
    name = "dev-user"
    tags = {
        Department = "Engineering"
    }
}
```

Step 2 --> Tag AWS Resources (e.g., S3 Buckets)

```
resource "aws_s3_bucket" "dev_bucket" {
    bucket = "engineering-bucket-123"
    tags = {
        Department = "Engineering"
    }
}
```

Step 3 --> Create IAM Policy with ABAC Rules for resource

```
data "aws_iam_policy_document" "s3_abac_policy" {
    statement {
        effect = "Allow"

        actions = [
        "s3:GetObject",
        "s3:PutObject",
        "s3:ListBucket"
        ]

        resources = [
        "arn:aws:s3:::*"
        ]

        condition {
        test     = "StringEquals"
        variable = "s3:ResourceTag/Department"
        values   = ["${aws_iam_user.developer.tags["Department"]}"]
        }
    }
}
```

Step 4 --> Attach Policy to IAM User (Now User 'dev-user' can only access buckets that have the same 'Department' tag)

```
resource "aws_iam_policy" "s3_abac" {
    name       = "ABACPolicy"
    policy     = data.aws_iam_policy_document.s3_abac_policy.json
}

resource "aws_iam_user_policy_attachment" "dev_user_policy" {
    user       = aws_iam_user.developer.name
    policy_arn = aws_iam_policy.s3_abac.arn
}
```

Q How do you secure pipeline identities accessing AWS resources?

Use IAM roles with limited session duration (session_duration = "PT4H")

Avoid long-lived credentials (max_session_duration = 14400)

Enable MFA for human access

Use Audit logging via CloudTrail to monitor access

Q What is a secure pattern for deploying to multiple AWS accounts using a CI/CD pipeline? / Use cross-account role assumption with trust policies?

If your Jenkins setup includes Folders, you can :

Create a folder. --- Move the job into the folder. --- Apply Folder-level authorization / Role-Based Plugin on target folder

Q How do you audit and validate access control in automated pipelines?

Use AWS Access Analyzer for policy validation

AWS Console -- IAM -- Add a New User -- Configure User Details and AWS access type -- Check “Access key and Check “Password -- Next -- Attach policies directly -- Add Tags -- -- Review and Create user

Now use IAM Access Analyzer to validate policy

IAM --> Policies --> Click on the name of the policy attached to your user ---> choos "Policy actions” dropdown and select “Validate policy”.

or

IAM > Users --> click on the user whcih you have created --> Permissions tab --> click on the policy name from policy list --> click “Validate policy” using Access Analyzer

Run IAM Access Advisor and IAM Policy Simulator

The IAM Access Advisor shows which AWS services a user has accessed and when

Q How do you rotate credentials and secrets automatically?

AWS Secrets Manager or SSM Parameter Store with automatic rotation.

AWS Console — Secrets Manager — Store a new secret -- Select secret type -- Provide secret key-value pairs (e.g., username, password) -- Click Next -- Secret name -- add description and tags -- Enable automatic rotation -- Set rotation interval (e.g., every 30 days) -- Click Next -- Click Store

Q How do you rotate credentials and secrets automatically using terraform?

Q How would you secure an S3 bucket?

Block public access settings ==> S3 Console -- Click on your bucket name. -- "Permissions" tab.-- "Block public access

Use bucket policies ==> S3 Console -- Click on your bucket name. -- Properties -- Permissions -- Bucket Policy

Enable encryption (SSE) ==> S3 Console -- Click on your bucket name. -- Properties -- Scroll down to "Default encryption

IAM policies for least privilege ==> Go to IAM > Users or Roles -- Assign policies on the bucket

Enable logging and monitoring

Q What is S3 pre-signed URL and how it is used?

A pre-signed URL is a URL that includes:

The bucket name and object key.

AWS access credentials (embedded securely).

An expiration time.

A signature

ex --> https://my-bucket.s3.amazonaws.com/docs/invoice.pdf?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=...

Q What is S3 event?

S3 event refers to a notification that is triggered by a specific action in an Amazon S3 bucket.

s3:ObjectCreated:* – Triggered when a new object is uploaded. Includes subtypes like Put, Post, Copy, and CompleteMultipartUpload
s3:ObjectRemoved:* – Triggered when an object is deleted. Includes Delete, DeleteMarkerCreated
s3:ObjectRestore:* – Triggered when an object is restored from Glacier.
s3:ReducedRedundancyLostObject – Triggered when an RRS object is lost.

Q How to create event in S3?

S3 Console -- your bucket name -- Properties -- Event notifications -- Create event notification
-- name , Event types: (e.g., All object create events) , Destination: Choose Lambda, SNS topic, or SQS queue
-- Click Save

Q How to create event in S3 using terraform?

```
provider "aws" {
    region = "us-east-1"
}

# --- 1. Create an S3 bucket ---

resource "aws_s3_bucket" "example" {
    bucket = "my-example-s3-bucket-12345"
}

# --- 2. Create a Lambda function --- / use Lambda function already exists

data "aws_lambda_function" "existing_lambda" {          ||
    function_name = "my-existing-lambda"                ||---- use Lambda function already exists
}

# --- 3. Allow S3 to invoke the Lambda function ---

resource "aws_lambda_permission" "allow_s3" {
    statement_id  = "AllowS3Invoke"
    action        = "lambda:InvokeFunction"
    function_name = data.aws_lambda_function.existing_lambda.function_name
    principal     = "s3.amazonaws.com"
    source_arn    = aws_s3_bucket.example.arn
}
```

```
# --- 4. Configure S3 Event Notification ---

resource "aws_s3_bucket_notification" "bucket_notification" {
    bucket = aws_s3_bucket.example.id

    lambda_function {
        lambda_function_arn = data.aws_lambda_function.existing_lambda.arn
        events              = ["s3:ObjectCreated:*"]
        filter_prefix       = "uploads/"
        filter_suffix       = ".jpg"
    }

    depends_on = [aws_lambda_permission.allow_s3]
}
```

Q create 3 instance with different aim using for_each loop?

```
variable "instances" {
    default = {
        instance1 = {
            aim = "web-server"
            ami = "ami-0c55b159cbfafe1f0"
            instance_type = "t2.micro"
        }
        instance2 = {
            aim = "database"
            ami = "ami-0c55b159cbfafe1f0"
            instance_type = "t2.medium"
        }
        instance3 = {
            aim = "cache"
            ami = "ami-0c55b159cbfafe1f0"
            instance_type = "t2.small"
        }
    }
}

resource "aws_instance" "example" {
    for_each = var.instances
    ami = each.value.ami
    instance_type = each.value.instance_type
    tags = {
        Name = each.key
        Aim  = each.value.aim
    }
}

output "instance_ids" {
    value = { for k, inst in aws_instance.example : k => inst.id }
}

output "instance_aims" {
    value = { for k, inst in aws_instance.example : k => inst.tags["Aim"] }
}
```

Q What are the components of vpc?

```
VPC
 |---Subnets (VPC IP range)              --- create ips
        |          |----- public ips
        |          |----- private ips
        |---Route Tables                  --- to access internet
        |          |----- Public subnets    → Route Table with NAT Gateway        <--------| ( Nat gateway help to
        |          |----- Private subnet    → Route Table with NAT Gateway access   ------|  access internet )
        |          |----- Private subnet    → without NAT gateway access
        |---Internet Gateway (IGW)
        |---NAT Gateway / NAT Instance
        |---Security Groups
        |           |----- ssh
        |           |----- http
        |           |----- tcp
        |---Network Access Control Lists (NACLs) --- to Control traffic in and out of subnets
        |---VPC Peering                          ---- Communication across two or more VPCs
        |---VPN Gateway                          ---- Secure connection between on-premise
        |---Elastic IP Addresses                                network and VPC over the internet
        |---DHCP Option Sets -                   --- Assign domain names, DNS servers, etc.
```

Q How Nat gateway work? / How a private subnet access internet?

The private instance can access the internet using Nat gateway, but the internet cannot access the private instance directly.
Nat gateway does not allow inbound connections from the internet to the private instance.
NAT Gateway, knows how to route the response, So the response from the internet comes back to the NAT Gateway, then to the private instance.

A private EC2 instance (e.g. backend EC2) wants to access the internet (e.g., to download a package)sends a request through route table. Route table routes (0.0.0.0/0) request to the NAT Gateway.
The request goes to the NAT Gateway, which lives in a public subnet.
NAT Gateway receives the traffic, and translates the source IP from the private EC2 IP (e.g., 10.0.2.15) to its own Elastic IP (e.g., 18.234.56.78) (Elastic IP attached to nat gateway)
Note ---> NAT Gateway + Elastic IP = One-Way Internet Access
Then NAT Gateway forwards the request to the Internet Gateway.
The response from the internet comes back to the NAT Gateway, then to the private instance.

Q Your private EC2 instance cannot access the internet, even though a NAT Gateway is set up in the public subnet.

What could be the issue?

The route table associated with the private subnet doesn't have a route to the NAT Gateway (no 0.0.0.0/0 to NAT GW).
NAT Gateway is not properly associated with the Elastic IP (required for internet access)
Network ACLs/Security Groups are blocking outbound traffic.
NAT Gateway is in Availability Zone that's different from private subnet and there's no route between them.

Q How can you reduce NAT Gateway data transfer charges?

Use VPC Endpoints (S3, DynamoDB): Avoid routing traffic through the NAT Gateway for AWS services.
Minimize unnecessary outbound internet traffic (e.g., OS updates, telemetry).
Aggregate data transfers (e.g., through a single instance or batch processing).

Q You set up a new NAT Gateway in a public subnet, but none of the private EC2 instances can reach the internet. What might be misconfigured?

NAT Gateway is in a public subnet (must have a route to the Internet Gateway).
Private subnet has a route to the NAT Gateway (0.0.0.0/0 → NAT GW).
Security Groups allow outbound traffic (usually default allows all).
The Elastic IP is associated and not released.
NACLs do not block the traffic.

Q How you Ensure NAT Gateway is in a Public Subnet and route to an Internet Gateway (IGW)?

Go to VPC Console → public Subnets where the NAT Gateway is deployed -- Check the Route Table -- Select the associated route table → Edit routes -- fill (Destination: 0.0.0.0/0 and Target: Internet Gateway) – save

Q How you will confirm Private Subnet Routes to NAT Gateway?

VPC Console → private Subnet which need to connect internet -- Route Table -- Ensure there is a route
(Destination: 0.0.0.0/0 → Target: NAT Gateway)

Q How you Ensure the private subnet has a security group which allowing outbound traffic?

EC2 Console → private EC2 Instances -- click Security Group -- Outbound rules (Type: HTTP, HTTPS, and Destination: 0.0.0.0/0)

Q How to ensure that an Elastic IP (EIP) is attached and not released for your NAT Gateway?

VPC Console -- NAT Gateways -- In the NAT Gateway details, look for the Elastic IP address

If Elastic IP address is not present, then NAT gateway is not attach to Elastic IP

To add Elastic ip

Open the EC2 Console -- choose Elastic IPs under Network & Security -- Allocate Elastic IP address -- Allocate
Then select newly created Elastic IP -- Click Actions → Associate Elastic IP address -- Choose NAT Gateway -- Click Associate.

Q You created two NAT Gateways in different Availability Zones (AZs) for high availability. Still, your private subnet traffic isn't failing over during AZ issues. Why?

Use multiple NAT Gateways (one per AZ) and assign AZ-specific route tables.

A private EC2 instance needs to download patches from the internet during maintenance. If there is no NAT Gateway set up patching cannot be done when required.

Q How can you enable temporary access?

Temporarily move the private instance to a public subnet and assign an Elastic IP.
Add a NAT Instance (manual, but quick if NAT Gateway not preferred).
Create a temporary NAT Gateway, route the subnet to it, then delete after use.

Q How to Move the Instance to a Public Subnet?

Stop the EC2 instance in private subnet
Create an AMI from the instance (right-click → Create Image)
Launch a new EC2 instance in a public subnet, using newly created AMI
Then do your work and then terminate the original instance if no longer needed.

Q A NAT Gateway is created, but it is still not passing any traffic. What should you check in the public subnet?

The subnet has a route to the Internet Gateway.
The NAT Gateway is associated with an Elastic IP.
The public subnet's route table must allow outbound access.
The NAT Gateway's ENI (Elastic Network Interface) is active and in the correct AZ.

Q Can NAT Gateway be used to allow inbound traffic to private subnets?

No, NAT Gateway only allows outbound internet traffic from private subnets.
For inbound access, use: Load Balancer in public subnet routing to private instances / Application Load Balancer.

Q How to configure Route table in AWS?

VPC Dashboard -- Route Tables > Create route table -- Enter a name and select your VPC -- Create route table
Select your new route table ---- click Edit routes ---- Add routes
|----- Add 0.0.0.0/0 → Internet Gateway (IGW)
|
|----- 10.0.2.0/24 → if other VPC Peered (the ip is for other VPC)|
---- Click Save-- Click Subnet Associations ---- Edit subnet associations ---- Select the subnets that should use this route table ---- Click Save

Q A host in a local network can communicate with other hosts in the same subnet but cannot reach any external websites. What could be the issue in the routing table?

The most likely issue is that the default route (0.0.0.0/0) is either missing or incorrectly configured in the routing table. This default route is essential for directing traffic destined for external networks e.g., the internet) to the default gateway.

Q For 192.168.1.0/24 via Router A and 192.168.1.0/25 via Router B which one has more specific (longer) subnet? Which route will be chosen, and why?

/25 is a more specific (longer) subnet than /24, because it uses more bits for the network portion and allows fewer hosts.

In IPv4, an IP address is 32 bits long (8bit + 8bit + 8bit + 8bit ---- for ip address)

/25 means the first 25 bits are used for the network portion. (11111111.11111111.11111111.10000000)

7 bits (32 - 25) for the host portion.

2^7 = 128 no of total IP addresses.

but for /24 -- total ip addresses is 2^8 = 256

In this case it will choose Router B because it is more specific (longer subnet mask).

Q In AWS, an EC2 instance in a private subnet can't access the internet, even though it has a NAT gateway. What should you check in the route table?

Verify the private subnet's route table has a route like:
0.0.0.0/0 → NAT Gateway ID
If this route is missing or incorrect, internet-bound traffic will not reach the NAT gateway.

Q What are the storage array you worked on?

I worked on AWS EBS, S3
io2 type EBS --> Critical business applications for databases like Oracle, SQL Server
gp3 type EBS --> For development/test environments / Web servers / App servers

Q You have a VPC with two subnets: one public and one private. You launch a web server in the public subnet and a database server in the private subnet. The web server needs internet access; the DB must not. How do you configure this?

Step 1 - Attach an Internet Gateway (IGW) to the VPC.

VPC Dashboard → Internet Gateways -> Create internet gateway → Name it -> Click Create internet gateway -> Select your new IGW → Click Actions → Attach to VPC -> Choose your VPC -> Click Attach internet gateway

Step 2 - Route the public subnet's traffic to the IGW via a route table.

VPC Dashboard → Route Tables -> Find associate it with your VPC or create a route table -> associate with public subnet
Select that route table → Go to Routes tab → Click Edit routes → Add route
Destination: 0.0.0.0/0
Target: Choose your Internet Gateway
Save the route
Go to the Subnet associations tab → Edit subnet associations → Select your public subnet → Save

Step 3 - Ensure the public subnet has an auto-assigned public IP or Elastic IP.

VPC Dashboard → Subnets → Select your public subnet -> Click Actions → Modify auto-assign IP settings -> select Enable auto-assign public IPv4 address → Save
or
EC2 Dashboard → Elastic IPs → Allocate Elastic IP address -> select Elastic IP address -> Allocate your EC2 instance in the public subnet

Step 4 - The private subnet has no route to the IGW — only to internal VPC traffic.

Go to Route Tables in the VPC dashboard -> Select the private subnet's route table -> ensure there's no route to 0.0.0.0/0 for no IGW route -> Has only the local route

Step 5 - Use security groups to allow HTTP/HTTPS to the web server and allow only internal traffic to the DB.

For public Subnet -> EC2 Dashboard → Security Groups → Create or select security group ->Inbound rules -> HTTP , HTTPS , SSH is enable

for private subnet -> EC2 Dashboard → Security Groups → Create or select security group ->Inbound rules -> TCP (specify your required port) -> No outbound

Q You have two VPCs in the same AWS region. You want instances in VPC-A to talk to instances in VPC-B securely. How do you do it?

Create a VPC Peering Connection between VPC-A and VPC-B from both sides.

VPC Dashboard -> click Peering Connections -> Create Peering Connection -> fill Requester VPC and Accepter VPC
After creation, go back to Peering Connections -> Find and Select the new connection -> Click Actions → Accept Request.

Update route tables in both VPCs to route traffic to each other's CIDR blocks via the peering connection from both side.

VPC dashboard, go to Route Tables -> Find the route table associated with VPC-A -> Click Edit Routes → Add Route

-> Destination: VPC-B's -> Target: Select the Peering Connection ID -> save

Ensure security groups allow traffic from the peer VPC's CIDR.

in VPC-A -> Go to Security Groups -> Select the Security Groups -> Click Inbound Rules → Edit Inbound Rules → Add Rule

-> Type: Custom TCP , Source: CIDR block of VPC-B (e.g., 10.1.0.0/16) -> save

Do the above all steps in VPC-B

Q You have a private subnet where EC2 instances need to access the internet for updates (e.g., apt-get), but you don't want them to be publicly accessible. How do you achieve this?

Launch a NAT Gateway in a public subnet.
Assign an Elastic IP to the NAT Gateway.
Update the route table of the private subnet to send 0.0.0.0/0 traffic to the NAT Gateway.
The NAT Gateway forwards requests to the internet and returns the responses, while instances remain private

Q What's the difference between a Security Group and a Network ACL in VPC?

NACLs to block IP ranges across an entire subnet; use SGs for app-specific access.
Go to VPC Dashboard → Network ACLs ; Go to EC2 Dashboard → Security Groups

Q You want your EC2 instances in private subnets to access S3 without going through the internet. How do you do that?

Create a VPC Endpoint of type Gateway for S3.
Select the appropriate route table(s) and subnets.
Add an entry in the route table:
Destination: S3 prefix list → Target: VPC Endpoint.
Now traffic to S3 stays within AWS and doesn't need a NAT Gateway or Internet Gateway.

Q You need to set up a custom VPC with high availability. What's a standard best-practice layout?

VPC with at least 2 public and 2 private subnets, spread across 2 Availability Zones.
Internet Gateway attached to VPC.
NAT Gateway in each public subnet for private subnet internet access.
Route tables per subnet type.
Use Security Groups, NACLs, and CloudWatch Logs for security and monitoring.

Q What is PreSigned URL for an S3 Object and how to Create a PreSigned URL for an S3 Object via AWS Console?

A Pre-Signed URL in Amazon S3 is a temporary, secure link that gives anyone limited access to an S3 object (file) without AWS credentials.

'S3' service -> Click on the bucket -> Select the Object -> Click on the Actions -> Share with a pre-signed URL -> Select the minutes/ hours -> click Generate Presigned URL

Example of a pre-signed url --> https://focus_allstate.s3.amazonaws.com/searchRetrive/